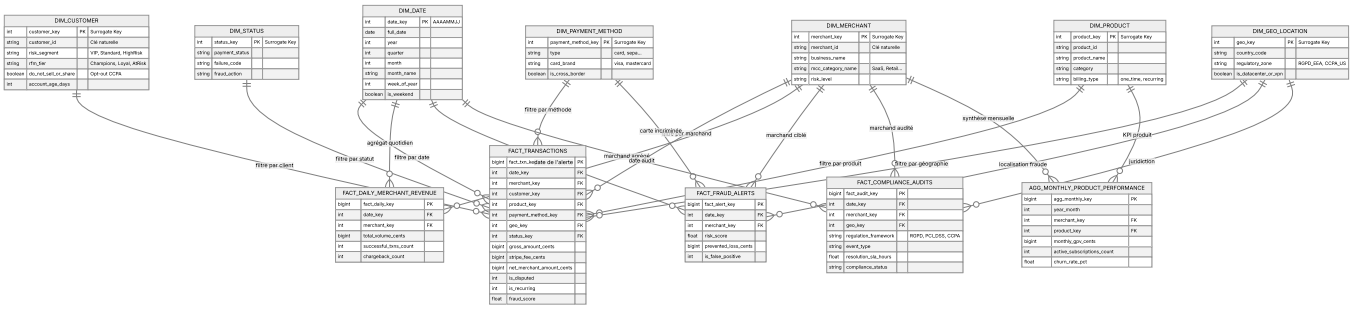
STRIPE AI INFRASTRUCTURE • CERTIFICATION BLOC 2 (RNCP 41993)

Modèle Analytique OLAP — Data Warehouse, Schéma en Étoile & Optimisations

Tandis que l'OLTP optimise les écritures unitaires (lignes), l'OLAP doit scanner des **centaines de millions de lignes** pour calculer des totaux, moyennes, segmentations clients, performances produits et rapports de conformité en quelques millisecondes. Le choix d'un **Schéma en Étoile (*Star Schema*)** sur **Amazon Redshift Serverless** s'impose pour Stripe : 1. **Simplicité & Rapidité des Jointures :** Chaque table de faits est reliée directement aux dimensions par des clés de substitution entières (`INT`), éliminant les cascades de jointures complexes du modèle 3NF. 2. **Optimisation pour le Stockage en Colonnes :** Redshift compresse chaque colonne indépendamment (ZSTD / AZ64) et ne lit depuis le disque que les colonnes sollicitées par la requête (ex : `SUM(gross_amount_cents)`). 3. **Clarté pour les Métiers & Outils BI :** Les analystes, auditeurs financiers/conformité et outils de DataViz (Metabase, Tableau) manipulent des concepts métiers intuitifs.

2. Diagramme du Schéma en Étoile OLAP

> *Fichier source de modélisation DBML :* `stripe_olap.dbml` & schéma haute résolution `stripe_olap_schema.png` (inclus dans le dossier livrable).



3. Détail des Tables de Faits, Audit & Granularité (Grain)

Le choix du **grain** (niveau de détail unitaire d'une ligne dans la table de faits) est le fondement de la modélisation dimensionnelle de Ralph Kimball :

Table de Faits / Synthèse	Grain Précis de la Table	Rôle Métier Principal	Type de Mesures
`FactTransactions`	**1 ligne = 1 transaction unitaire** exécutée sur Stripe.	Analyse fine de rentabilité, taux de refus, marges d'interchange, part des abonnements (`is_recurring`) et audit de latence IA.	**Additives :** montants en centimes, commissions. **Non-additives :** score de risque, latence en ms.
`FactDailyMerchantRevenue`	**1 ligne = 1 marchand pour 1 jour calendaire.**	Accélération massive des dashboards marchands (volumes quotidiens, taux de succès, relances réussies), relevés financiers.	**Strictement additives :** volumes cumulés, compteurs d'opérations, chargebacks.
`FactFraudAlerts`	**1 ligne = 1 décision de blocage ou d'alerte Radar.**	Suivi de l'efficacité des modèles ML, pertes financières évitées (*prevented loss*), suivi des faux positifs.	**Additives :** montants protégés. **Semi-additives :** distance vectorielle, score moyen.
`FactComplianceAudits`	**1 ligne = 1 événement réglementaire ou de contrôle.**	Traçabilité des demandes RGPD (effacement Art. 17), *Opt-Out* CCPA, contrôles PCI-DSS (rotation KMS, blocage PAN) et respect des SLA légaux.	**Additives :** `records_affected` . **Non-additives :** `resolution_sla_hours` .
`AggMonthlyProductPerformance`	**1 ligne = 1 couple Marchand / Produit par mois.**	Suivi de la **performance produit**, adoption du récurrent, taux de renouvellement Smart Retries et attrition mensuelle.	**Additives :** GPV mensuel, abonnements actifs. **Calculées :** taux de renouvellement et de churn.

4. Détail des Tables de Dimensions & Stratégie SCD

Les dimensions contiennent le contexte textuel et descriptif permettant de découper (*slice and dice*) les faits :

- **Clés de Substitution (*Surrogate Keys*) :** Toutes les dimensions utilisent des clés entières artificielles (`merchant_key INT` , `product_key INT` , `customer_key INT`), 4 à 5 fois plus rapides sur Redshift que les jointures sur chaînes `VARCHAR` et compatibles avec l'historisation **SCD Type 2** (suivi historique des changements de niveau de risque d'un marchand sans altérer le passé).
- **`DimCustomer` (Segmentation & CCPA) :** Intègre la segmentation comportementale **RFM** (`rfm_tier` : *Champions*, *Loyal*, *AtRisk*, *Churned*), le segment de risque (`VIP` , `Standard` , `HighRisk`) et le marqueur de conformité `do_not_sell_or_share` filtré automatiquement lors des analyses marketing.

- `DimProduct` **(Performance Produit) :** Permet de ventiler le chiffre d'affaires et l'attrition par produit catalogue et modèle de facturation (`one_time` vs `recurring`).
- `DimDate` **&** `DimGeoLocation` **:** Clé temporelle structurée `YYYYMMDD` évitant les fonctions de troncature coûteuses en requête, et drapeaux géographiques `regulatory_zone` (`RGPD_EEA` , `CCPA_US`) et `is_datacenter_or_vpn` .

---

5. Exemples de KPI Métiers Calculés via le Modèle OLAP

1. **Volume d'Affaires Brut (GPV - Gross Payment Volume) :** $$\text{GPV} = \sum (\text{FactTransactions.gross_amount_cents}) \quad \text{où } \text{status} = \text{'succeeded'}$$
2. **Taux de Litiges / Chargebacks (Règle des 1% Card Scheme) :** $$\text{Chargeback Rate} = \frac{\sum \text{FactTransactions.is_disputed}}{\text{COUNT} (\text{FactTransactions.fact_txn_key})}$$ *Seuil critique Visa/Mastercard fixé à 0.9% : au-delà, le marchand est placé sous surveillance renforcée.*
3. **Marge Nette Stripe par Catégorie Métier (Take Rate Net) :** $$\text{Net Take Rate} = \frac{\sum (\text{stripe_fee_cents} - \text{interchange_fee_cents})}{\sum \text{gross_amount_cents}}$$
4. **Pertes Évitées par l'IA Radar (Prevented Fraud Volume) :** $$\text{Total Prevented} = \sum (\text{FactFraudAlerts.prevented_loss_cents}) \quad \text{où } \text{action_taken} = \text{'block'}$$

---

6. Stratégie d'Optimisation : Jointures à Grande Échelle, Séries Temporelles & Pré-Agrégations

Pour garantir des temps de réponse inférieurs à la seconde sur des tables dépassant le milliard de lignes sur **Amazon Redshift Serverless** :

6.1. Gestion des Jointures à Grande Échelle (Distribution Colocalisée)

- **Colocalisation par** `DISTKEY(merchant_key)` **:** Les tables massives (`FactTransactions` , `FactDailyMerchantRevenue` , `FactFraudAlerts`) ainsi que `DimMerchant` et `DimCustomer` utilisent `DISTSTYLE KEY` **sur** `merchant_key` . Les lignes d'un même marchand résident physiquement sur le même nœud de calcul Redshift, éliminant tout transfert réseau (*network shuffle* / *broadcast*) lors des jointures à grande échelle.
- **Réplication des Petites Dimensions (** `DISTSTYLE ALL` **) :** Les dimensions compactes (`DimDate` , `DimProduct` , `DimPaymentMethod` , `DimGeoLocation` , `DimStatus`) sont intégralement répliquées sur chaque nœud de calcul, rendant leurs jointures locales et instantanées.

6.2. Optimisation des Séries Temporelles & Sous-Requêtes

- `COMPOUND SORTKEY(date_key, merchant_key)` **:** Les données sont physiquement triées par ordre chronologique sur disque. Chaque bloc de stockage (1 Mo) maintient ses métadonnées *Min/Max Zone Maps* : une requête sur les 30 derniers jours ignore 95%+ des blocs disques sans même les lire (*block pruning*).
- **Fonctions de Fenêtrage & CTE :** Les analyses de séries temporelles (moyennes mobiles sur 7/30 jours, comparaisons *Month-over-Month*) privilégient les **CTE matérialisées** et les fonctions

analytiques `SUM(...) OVER (PARTITION BY merchant_key ORDER BY date_key ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` à la place des sous-requêtes corrélées imbriquées ($O(N)$ contre $O(N^2)$).

6.3. Plan de Pré-Agrégations & Vues Matérialisées (Quotidien, Hebdomadaire, Mensuel)

Afin d'éviter de re-scanner `FactTransactions` pour les tableaux de bord récurrents, trois niveaux de synthèse automatisée sont maintenus :

Niveau Temporel	Objet OLAP	Rafraîchissement	Cas d'Usage Principal
Quotidien	`FactDailyMerchantRevenue`	Incrémental nocturne (dbt/Airflow 02h UTC)	Cockpits marchands (GPV jour, rejets, virements).
Hebdomadaire	`mv_weekly_merchant_fraud_kpi` (*Materialized View*)	Automatique (`AUTO REFRESH YES` Redshift)	Suivi hebdomadaire des chargebacks Visa/MC et pertes évitées Radar (< 150 ms).
Mensuel	`AggMonthlyProductPerformance` & `mv_monthly_revenue_segment`	Mensuel / Incrémental dbt	Reporting financier CFO, performance produit et analyse de cohortes/churn.

```
-- Exemple d'implémentation Redshift : Vue Matérialisée Hebdomadaire Auto-Rafraîchie
CREATE MATERIALIZED VIEW stripe_olap.mv_weekly_merchant_fraud_kpi
BACKUP NO
AUTO REFRESH YES AS
SELECT
    d.year,
    d.week_of_year,
    m.merchant_key,
    m.mcc_category_name,
    COUNT(f.fact_txn_key) AS txns_count,
    SUM(f.gross_amount_cents) AS weekly_gpv_cents,
    SUM(f.is_disputed) AS dispute_count,
    AVG(f.fraud_score) AS avg_fraud_score
FROM stripe_olap.fact_transactions f
JOIN stripe_olap.dim_date d ON f.date_key = d.date_key
JOIN stripe_olap.dim_merchant m ON f.merchant_key = m.merchant_key
GROUP BY d.year, d.week_of_year, m.merchant_key, m.mcc_category_name;
```